OdyCard — AI Waiter for Sree Annapoorna

Project Summary & Plan (Idea Stage)

Last updated: 9 June 2026

---

1. What this is

A **custom, text-based AI waiter chatbot** built specifically for **Sree Annapoorna** (the Coimbatore pure-veg chain). It is **not** the broader OdyCard menu app, **not** an ordering system, and **not** a self-serve owner platform. It is bespoke software tailored for one client.

The customer flow:

> *Customer sits down → scans a QR code on the table → a chatbot page opens in their phone browser → they chat with **Ody** (the AI waiter) to decide what to eat → Ody recommends, answers questions, handles banquet/info queries → **the customer then places their order with a human waiter.***

Ody does what a good waiter does — advise, recommend, explain — but does **not** take the order itself.

Hard constraints: - **Text-only.** Voice is deferred to the future (cost + Annapoorna didn't ask for it). - **Lean / low-cost.** This is idea stage; we don't risk much money yet. - **No menu display, no dish cards, no ordering, no owner onboarding UI.** Stripped out to stay focused and cheap.

---

2. The non-negotiable: grounding

Ody can only answer from **Annapoorna's real menu and information**. The number-one risk with a real brand is hallucination — Ody inventing a dish or a wrong price in front of an Annapoorna diner is the fastest way to lose the customer and damage the brand.

Where the menu data comes from: Since this is a single bespoke client, **our team loads Annapoorna's menu once** on the backend (from their existing menu card / PDF). There is **no owner upload tool** — that whole flow is deleted. "No menu uploading by owners" means no self-serve UI, *not* that Ody operates blind.

Data we need to load per dish: name, price, ingredients, veg/Jain (no onion-garlic) flags, allergens, a short description, and **margin/cost** (for smart recommending). The recommendations are only as good as this data, so this one-time enrichment is the biggest hidden task.

We reuse the existing stack: Next.js / React / TypeScript / Tailwind, **Firebase Firestore** (menu data already lives here), Cloudinary, deployed on Vercel. QR generation already exists.

---

3. What Ody does (the jobs)

1. **Acts as a waiter** — intelligent, personalised menu recommendations.
2. **Answers dish questions** — ingredients, spice level, veg/Jain, "what's in it."
3. **Banquet hall enquiries** — info + lead capture + handoff to a human.
4. **Restaurant info** — hours, location, parking, AC, reservations, etc.
5. **Honest margin-aware recommending** — gently favours profitable dishes *without* the customer ever feeling pushed.
6. **Live availability awareness** — never recommends a sold-out dish.
7. **Customer feedback** — captured during/after the chat.
8. **Escalation to a human** — for complaints, allergy-critical questions, and complex banquet quotes.

---

4. Honest margin-aware recommending (the delicate one)

The principle (your own instinct): the moment a customer *feels* pushed toward expensive food, they stop trusting the bot — and a manipulative bot is a reputation risk for Annapoorna, not an asset. So we do **not** build deception (no hiding cheaper options, no dark patterns). That backfires commercially anyway.

How it actually works: - When several dishes genuinely fit what the customer wants, Ody leans toward the ones that are **both well-rated AND higher-margin**. - It frames them by **real value** — popular, chef's special, "pairs well with your order" — **never by price**. - It **never disparages or hides** the cheaper option. - Margin is a **gentle tiebreaker among genuinely good suggestions** — invisible to the customer because every suggestion is actually good.

Requirement: Annapoorna must share **cost/margin data per dish**. No margin data → this feature is dead on arrival. (Open item to confirm with them.)

---

5. Banquet handling

Banquet splits into two halves:

- **Informational half** ("do you have a hall, what's the capacity, do you do veg catering, what packages") → handled by **Alpha**, pre-seeded with Annapoorna's real details, written in a **warm tone** (these are wedding/event enquiries).

- **Transactional half** ("is the hall free on Aug 15, quote me for 250 guests, I want to book") → Ody can't answer this (dynamic dates, custom quotes need a human). It triggers **lead capture** (name, phone, date, headcount) → handed to the manager.

Priority note: For banquet, **don't optimise for tokens** — it's low-volume but high-value. One captured wedding lead beats thousands of saved dish-price lookups. If Alpha lacks a pre-written answer, fall back to **Beta** so it never dead-ends on a high-value enquiry.

Open item: does Annapoorna want hall pricing shown publicly, or just "leave your details and we'll call you"?

---

6. The two-layer engine: Alpha & Beta

A router design (your friend's idea) to keep AI/token costs low.

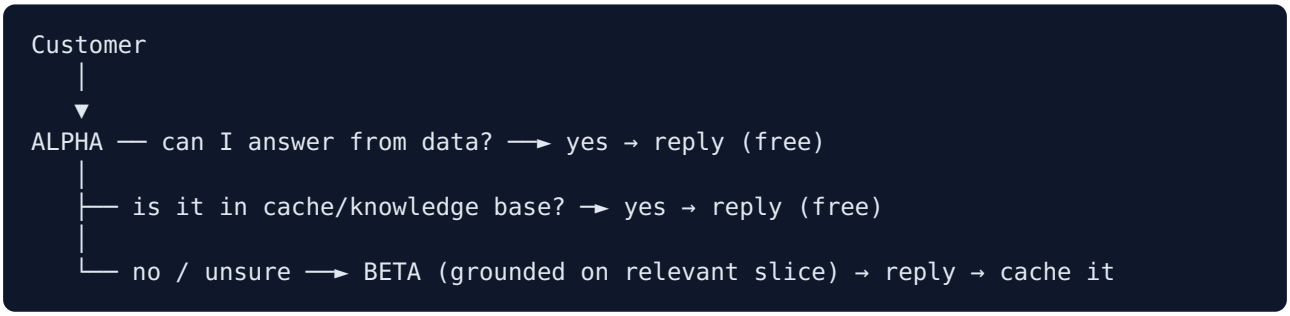
Alpha — the cheap, fast layer (no LLM generation)

- Answers **simple structured queries via a live database lookup**: price, availability, timing, veg/Jain flags. **Zero tokens.**
- **Pre-seeded** with restaurant info + banquet FAQ (we author these up front).
- Serves **stable conversational answers** from a knowledge base that grows from Beta (see §7).
- Uses **intent + dish detection via embeddings** (cheap, meaning-based) — not brittle keyword matching — so it handles **Tamil / Tanglish / English**.
- **Escalates to Beta when unsure.** Critically, it **never short-circuits anything safety- or trust-critical** (allergy questions, complaints, banquet bookings) — those always go to Beta or a human.

Beta — the AI layer

- **Gemini Flash** (or ChatGPT mini) — cheap, fast, handles Tamil.
- Handles complex / conversational queries.
- **Grounded on just the relevant menu slice** that Alpha hands it (smaller context = fewer tokens, even when escalating).

The flow



---

7. "Alpha learns from Beta" — done as retrieval, NOT model training

The goal — Alpha gets smarter over time so Beta is called less — is right. The **implementation** is the important part.

We do NOT train/fine-tune our own model now. Reasons (none of them are about owning GPUs — you can rent those on AWS): - **No data yet.** Fine-tuning needs hundreds–thousands of real Q&A examples; Annapoorna isn't live, so we have zero conversations. Can't train on chats that haven't happened. - **Menu facts change.** A trained model bakes knowledge in; the moment a price changes it's confidently wrong and you'd retrain forever. Retrieval updates instantly — edit one record. - **For changing facts, retrieval is more *correct* than training**, not just cheaper. (Fine-tune for *behaviour/style*; retrieve for *facts that change*.)

The cheap version that gives the same result: - When Beta answers something new, **save the question→answer pair** in a knowledge base. - Next time a *similar* question comes, Alpha **matches it via embeddings** and serves the stored answer for free. - The store grows daily → Beta gets called less and less.

Guardrails on the learning: - **Validate with Beta only during a short learning phase or on a random sample** — never on every query (that would kill the savings). - **Never cache volatile facts** (price, availability, timing) — always live DB lookup. Only cache **stable explanatory answers**, and wipe them when the menu changes. - **Thumbs-up gates learning:** only promote Beta answers that got a into Alpha's knowledge base; never cache answers.

When training *does* make sense → Phase 3: once we've collected thousands of real Beta transcripts, we could fine-tune a small model (on AWS/Bedrock/Vertex — rented/managed, not owned hardware) to make Alpha cheaper still. Only when there's both data and volume to justify it.

---

8. Sign-in & sessions

- **Optional sign-in. Google only for v1** (free). **Phone OTP is deferred** — SMS costs money per message and adds up at Annapoorna footfall.
- **Signed-in** → personalised and remembered across visits.
- **Anonymous** → can chat fully, but **ephemeral**: refresh = fresh chat, not remembered on their screen.
- **But every conversation is logged server-side** (anonymised, under a throwaway session ID) — *including* anonymous ones — for mining. "Forgotten on the front-end" and "captured on the back-end" are independent.
- **Privacy:** include a small disclosure ("chats may be used to improve recommendations") and keep mined data **aggregated / anonymised**. A real brand will care.

---

9. Interest signals (since there's no ordering)

With no ordering feature, Ody won't know what customers actually **bought**, so we substitute **interest signals**:

- **Thumbs up/down on Ody's replies** = a measure of **the bot's quality** (for us). Also **gates Alpha's learning** (§7).
- **In-chat hearts on dishes + an "explore more" option** = **food-preference / interest** signal.
- **Implicit signals** — *what the customer asked about and which suggestions they engaged with* — are the **richest and free**. This is the same conversation data the miner reads.

Honesty about the data: these show *interest*, not *purchase* (someone hearts a dessert but orders a dosa). So returning-customer lines are framed around **exploring**, e.g. "*Last time you were checking out the tiffin items — want to explore some chaat today?*" — truthful given the data.

UX caution: don't over-ask. Too many toggles (rate this, heart that, thumbs, explore) and people ignore all of them. Keep explicit asks minimal; lean on implicit conversation data.

(Note: the old plan to reuse favourites/eat-later is moot now — those were tied to the menu display, which is gone. Signals now come from the conversation + in-chat toggles.)

---

10. The data flywheel & mining

With no menu browsing, **the conversation is the only digital record of customer demand** — so mining it is central, not optional.

- **Log all conversations** during the day → run **one batch AI job (e.g. nightly)** that reads them and extracts structured insights. **Not live AI mining** — that's too expensive. Real-time AI is only for actual customer replies.
- **What to extract** (one category is pure gold):
- **Demand gaps** — dishes customers asked for that we don't offer or that were sold out. ("23 people asked for filter-coffee combos we don't list.") Direct, actionable demand data.
- Most-asked dishes
- Dietary requests (Jain / vegan)
- Price sensitivity
- Cuisine cravings
- Banquet enquiries
- Complaint themes
- Peak-time patterns

This feeds **(a)** better recommendations and **(b)** the owner dashboard.

10a. Google reviews — cold-start data source

Before launch we have zero conversation data, so Ody won't know what the crowd actually *loves*. **Annapoorna's existing Google reviews (old + new) fix this.**

- **How we get them — the clean, free, legal way:** via the **Google Business Profile API**. Annapoorna owns their listing, so they add us (or a service account) as a manager and we pull **all** their reviews legitimately — historical *and* a live feed of new ones.
- **What to avoid:** scraping Google Maps directly (violates Google's terms). Third-party aggregators (Outscraper/SerpApi) exist but sit in a grey area — unnecessary, since owner access is cleaner. The *Places* API only returns ~5 reviews, so it's not usable here.
- **Reviews aren't dish-level:** a review is free text about the whole restaurant. So we run **one AI batch pass** (Gemini Flash) to extract structured data — dish mentioned, sentiment, recurring themes. This is the *same* batch-mining pipeline as the chats; Google reviews are simply a second input.
- **Output** feeds the dashboard and gives Ody a soft sense of "what people love here."

Caveats (treat reviews as a soft prior, not ground truth): - Biased and sparse — people review when thrilled or furious, and most don't name dishes. - Can be stale — weight recent reviews more. - **Ody must never quote a review or claim "rated 4.8 on Google"** — internal weighting + dashboard only, never a customer-facing claim. - **Multi-outlet:** each Annapoorna branch has its own Google listing — keep each branch's review data separate (one branch's slow service isn't another's).

---

11. Owner / Manager dashboard

A dashboard for Annapoorna's owner/manager showing: - What customers like and want; **demand gaps** - Most common questions - What Ody recommended vs. what customers engaged with - Complaint themes - Banquet leads

This is the bridge to a future **"AI Manager"** (business-insights) product.

---

12. Feedback routing

Route customer feedback **by sentiment**: - **Negative** → flag the manager **fast**, while the customer is still in the restaurant (in-the-moment service recovery). - **Positive** → nudge toward a public review.

---

13. Safety & guardrails

- **Allergens / dietary = health-critical.** Ody answers from real data and **refuses to guess**; for a true allergy it escalates to a human ("let me get a staff member to confirm"). Getting this wrong is a liability.

- **Human escalation path** for complaints, complex banquet, allergy.
- **On-topic + jailbreak resistance** — keep Ody from being baited into saying things off-brand under Annapoorna's name.

---

14. Availability mechanism (the one bit of staff input still needed)

Even without an owner menu-tool, **something must mark dishes sold-out**, or Ody recommends unavailable food (the trust-killer). Minimal options: - A dead-simple staff page with on/off toggles, **or** - A daily/WhatsApp update that someone on our side enters.

Doesn't need to be fancy, but it can't be nothing.

---

15. Hosting & domain

- Cheapest: a **subdomain on our side** (e.g. `annapoorna.odysra.com`) — full control, zero setup cost.
- Can move to Annapoorna's own domain later if they want. Don't let this block the build.

---

16. Cost discipline (summary)

- **Beta = Gemini Flash** (or ChatGPT mini) — cheap models.
- **Alpha free-path** answers simple queries with zero tokens.
- **Semantic cache** for repeated complex questions.
- **Batch (not live)** conversation mining.
- **Google-only auth** — no SMS cost.
- **Inject only the relevant menu slice** to Beta.

---

17. Phasing

Phase 1 — Annapoorna pilot (lean v1) Chat UI + Alpha/Beta engine grounded on the once-loaded menu; banquet info + lead capture; restaurant info; honest-margin recommends; sold-out toggle; conversation logging; thumbs/hearts; a basic owner dashboard; optional Google sign-in. Hosted on a subdomain.

Phase 2 Phone OTP + deeper personalisation; richer dashboard; mature semantic cache; automated feedback routing.

Phase 3 Fine-tune a small model from real transcripts (cloud); possibly voice; AR dish view; multi-outlet; full "AI Manager."

---

18. Open decisions to confirm with Annapoorna

1. **Will they hand over cost/margin data?** (No margin data → the smart-push feature is dead.)
2. **Banquet pricing public, or "leave details and we'll call you"?**
3. **Domain — theirs or ours?**
4. **Who updates availability, and how?**
5. **Menu source** (their card/PDF) for the one-time load.
6. **Pilot scope — one outlet or the whole chain?** (Affects Google-review ingestion: chain-wide = multiple listings, kept separate per branch.)
7. **Google Business Profile access** — will Annapoorna add us as a manager so we can pull their reviews?

This document captures the current shared understanding. It's a living plan — expect it to keep evolving as Annapoorna's requirements firm up.